

Understanding Time Complexity with Simple Examples

Last Updated : 28 Aug, 2025



A lot of students get confused while understanding the concept of time complexity, but in this article, we will explain it with a very simple example.

Q. Imagine a classroom of 100 students in which you gave your pen to one person. You have to find that pen without knowing to whom you gave it.

Here are some ways to find the pen and what the O order is.

- **$O(n^2)$:** You go and ask the first person in the class if he has the pen. Also, you ask this person about the other 99 people in the classroom if they have that pen and so on,
This is what we call $O(n^2)$.
- **$O(n)$:** Going and asking each student individually is $O(N)$.
- **$O(\log n)$:** Now I divide the class into two groups, then ask: "Is it on the left side, or the right side of the classroom?" Then I take that group and divide it into two and ask again, and so on. Repeat the process till you are left with one student who has your pen. This is what you mean by $O(\log n)$.

I might need to do:

- The **$O(n^2)$** searches if **only one student knows on which student the pen is hidden.**
- The **$O(n)$** if **one student had the pen and only they knew it.**
- The **$O(\log n)$** search if **all the students knew**, but would only tell me if I guessed the right side.

*The above **O** -> is called Big - O which is an asymptotic notation. There are other asymptotic notations like theta and Omega.*

NOTE: We are interested in the rate of growth over time with respect to the inputs taken during the program execution.

Is the Time Complexity of an Algorithm/Code the same as the Running/Execution Time of Code?

The Time Complexity of an algorithm/code is **not** equal to the actual time required to execute a particular code, but the **number of times a statement executes**. We can prove this by using the [time command](#).

For example: Write code in C/C++ or any other language to find the maximum between N numbers, where N varies from 10, 100, 1000, and 10000. For Linux based operating system (Fedora or Ubuntu), use the below commands:

*To compile the program: **gcc program.c -o program***

*To execute the program: **time ./program***

You will get surprising results i.e.:

- For N = 10: you may get 0.5 ms time,
- For N = 10,000: you may get 0.2 ms time.
- Also, you will get different timings on different machines. Even if you will not get the same timings on the same machine for the same code, the reason behind that is the current network load.

So, we can say that the **actual time required to execute code is machine-dependent** (whether you are using Pentium 1 or Pentium 5) and also it considers network load if your machine is in LAN/WAN.

What is meant by the Time Complexity of an Algorithm?

Now, the question arises if time complexity is not the actual time required to execute the code, then what is it?

The answer is:

*Instead of measuring actual time required in executing each statement in the code, **Time Complexity considers how many times each statement executes**.*

Example 1: Consider the below simple code to [print Hello World](#)

C++

C

Java

Python

C#

JavaScript

```
#include <iostream>
using namespace std;
```

```
int main()
{
    cout << "Hello World";
    return 0;
}
```

```
// This code is contributed by vikash36905.
```

Output

Hello World

Time Complexity: In the above code “Hello World” is printed only once on the screen.

So, the time complexity is **constant: $O(1)$** i.e. every time a constant amount of time is required to execute code, no matter which operating system or which machine configurations you are using.

Auxiliary Space: $O(1)$

Example 2:

C++

C

Java

Python

C#

JavaScript

```
#include <iostream>
using namespace std;
```

```
int main()
{
    int i, n = 8;
    for (i = 1; i <= n; i++) {
        cout << "Hello World !!!\n";
    }
    return 0;
}
```

```
// This code is contributed by vikash36905.
```

Output

```
Hello World !!!  
Hello World !!!  
Hello World !!!  
Hello World !!!  
Hello World !!!  
Hello World !!!  
Hello World !!!  
Hello World !!!
```

Time Complexity: In the above code “Hello World !!!” is printed only **n times** on the screen, as the value of n can change.

So, the time complexity is **linear: $O(n)$** i.e. every time, a linear amount of time is required to execute code.

Auxiliary Space: $O(1)$

Example 3:

C++

C

Java

Python

C#

JavaScript

```
#include <iostream>  
using namespace std;  
  
int main()  
{  
  
    int i, n = 8;  
    for (i = 1; i <= n; i=i*2) {  
        cout << "Hello World !!!\n";  
    }  
    return 0;  
}
```

```
// This code is contributed by Suruchi Kumari
```

Output

```
Hello World !!!  
Hello World !!!  
Hello World !!!  
Hello World !!!
```

Time Complexity: $O(\log_2(n))$

Auxiliary Space: $O(1)$

Example 4:

C++

C

Java

Python

C#

JavaScript

```
#include <iostream>  
#include <cmath>  
using namespace std;  
  
int main()  
{  
  
    int i, n = 8;  
    for (i = 2; i <= n; i=pow(i,2)) {  
        cout << "Hello World !!!\n";  
    }  
    return 0;  
}  
  
// This code is contributed by Suruchi Kumari
```

Output

```
Hello World !!!  
Hello World !!!
```

Time Complexity: $O(\log(\log n))$

Auxiliary Space: $O(1)$

How To Find The Time Complexity Of An Algorithm?

Now let us see some other examples and the process to find the time complexity of an algorithm:

Example: Let us consider a model machine that has the following specifications:

- Single processor
- 32 bit
- Sequential execution
- 1 unit time for arithmetic and logical operations
- 1 unit time for assignment and return statements

Q1. Find the Sum of 2 numbers on the above machine:

For any machine, the pseudocode to add two numbers will be something like this:

C++ C Java Python C# JavaScript

```
// Pseudocode : Sum(a, b) { return a + b }
#include <iostream>
using namespace std;

int sum(int a,int b)
{
    return a+b;
}

int main() {
    int a = 5, b = 6;
    cout<<sum(a,b)<<endl;
    return 0;
}

// This code is contributed by akashish__
```

Output

11

Time Complexity:

- The above code will take 2 units of time(constant):

- one for arithmetic operations and
- one for return. (as per the above conventions).
- Therefore total cost to perform sum operation (**Tsum**) = 1 + 1 = 2
- **Time Complexity = O(2) = O(1)**, since 2 is constant

Auxiliary Space: O(1)

Q2. Find the sum of all elements of a list/array

The pseudocode to do so can be given as:

C++

C

Java

Python

C#

JavaScript

```
#include <iostream>
using namespace std;

int list_Sum(int A[], int n)

// A->array and
// n->number of elements in array
{
    int sum = 0;
    for (int i = 0; i <= n - 1; i++) {
        sum = sum + A[i];
    }
    return sum;
}

int main()
{
    int A[] = { 5, 6, 1, 2 };
    int n = sizeof(A) / sizeof(A[0]);
    cout << list_Sum(A, n);
    return 0;
}

// This code is contributed by akashish__
```

Output

14

To understand the time complexity of the above code, let's see how much time each statement will take:

C++

C

Java

Python

C#

JavaScript

```

int list_Sum(int A[], int n)
{
    int sum = 0;    // cost=1 no of times=1
    for(int i=0; i<n; i++)    // cost=2 no of times=n+1 (+1 for the end false
condition)
        sum = sum + A[i] ;    // cost=2 no of times=n
    return sum ;    // cost=1 no of times=1
}

```

Therefore the total cost to perform sum operation

$$T_{sum} = 1 + 2 * (n+1) + 2 * n + 1 = 4n + 4 = C1 * n + C2 = O(n)$$

Therefore, the time complexity of the above code is **O(n)**

Q3. Find the sum of all elements of a matrix

For this one, the complexity is a polynomial equation (quadratic equation for a square matrix)

- Matrix of size $n \times n \Rightarrow T_{sum} = a.n^2 + b.n + c$
- Since T_{sum} is in order of n^2 , therefore **Time Complexity = $O(n^2)$**

C++

Java

Python

C#

JavaScript

```

#include <iostream>
using namespace std;

int main()
{
    int n = 3;
    int m = 3;
    int arr[][3]
        = { { 3, 2, 7 }, { 2, 6, 8 }, { 5, 1, 9 } };
    int sum = 0;

    // Iterating over all 1-D arrays in 2-D array
    for (int i = 0; i < n; i++) {

        // Printing all elements in ith 1-D array
        for (int j = 0; j < m; j++) {

```



```

        // Printing jth element of ith row
        sum += arr[i][j];
    }
}
cout << sum << endl;
return 0;
}
// contributed by akashish__

```

Output

43

Time Complexity: $O(n*m)$

The program iterates through all the elements in the 2D array using two nested loops. The outer loop iterates n times and the inner loop iterates m times for each iteration of the outer loop. Therefore, the time complexity of the program is $O(n*m)$.

Auxiliary Space: $O(n*m)$

The program uses a fixed amount of auxiliary space to store the 2D array and a few integer variables. The space required for the 2D array is nm integers. The program also uses a single integer variable to store the sum of the elements. Therefore, the auxiliary space complexity of the program is $O(nm + 1)$, which simplifies to $O(n*m)$.

In conclusion, the time complexity of the program is $O(nm)$, and the auxiliary space complexity is also $O(nm)$.

So from the above examples, we can conclude that the time of execution increases with the type of operations we make using the inputs.

[embed]<https://www.youtube.com/watch?v=KXAbAa1mieU%5B/embed%5D>

How to Compare Algorithms?

To compare algorithms, let us define a few objective measures:

- **Execution times:** *Not a good measure as execution times are specific to a particular computer.*
- **The number of statements executed:** Not a good measure, since the number of statements varies with the programming language as well as the style of the individual programmer.
- **Ideal solution:** Let us assume that we express the running time of a given algorithm as a function of the input size n (i.e., $f(n)$) and compare these different functions corresponding to running times. This kind of comparison is independent of machine time, programming style, etc.
Therefore, an ideal solution can be used to compare algorithms.

Related articles:

- [Time Complexity and Space Complexity](#)
- [Analysis of Algorithms | Set 1 \(Asymptotic Analysis\)](#)
- [Analysis of Algorithms | Set 2 \(Worst, Average and Best Cases\)](#)
- [Analysis of Algorithms | Set 3 \(Asymptotic Notations\)](#)
- [Analysis of Algorithms | Set 4 \(Analysis of Loops\)](#)
- [Analysis of Algorithm | Set 5 \(Amortized Analysis Introduction\)](#)
- [Miscellaneous Problems of Time Complexity](#)
- [Practice Questions on Time Complexity Analysis](#)
- [Knowing the complexity in competitive programming](#)

Calculating Time Complexity

 Comment

More info 

Advertise with us

Explore

DSA Fundamentals



Data Structures



Algorithms



Advanced



Interview Preparation



Practice Problem



Do Not Sell or Share My Personal Information